

Enhancing Context Sensitivity Selection Using Non-trivial Features and Reinforcement Learning

Nancy Jia, University of Illinois at Urbana-Champaign

December 2023

1 Introduction

For this project, we looked at an already-existing pointer analysis framework, *Scalar* [4], that allows the performance of context-sensitive analysis on Java programs. We first extended the Soufflé and Groovy parts of the framework to allow *Scalar* to support more context sensitivity analysis options: 1-object sensitivity and 3-object sensitivity. We then introduced a notion of “precision importance” for context-sensitive pointer analysis. We implemented an evaluation of a function’s precision importance, based on its contribution to the size of large points-to set through return variables and input parameters, and used this evaluation for calculating the time-precision trade-off for analyzing a function. We also modified the framework to be amenable to Reinforcement Learning, which will be trained for decision making on context-sensitive analysis options for functions in a program. The key to support Reinforcement Learning on pointer analysis was to modify the existing framework to function incrementally, performing analysis on one function at a time while allowing the Reinforcement Learning agent to access the analysis results at any point. This

```

1  public class Person {
2      String cloth = "clean";
3      String mood = "neutral";
4
5      void laundry(Person p) {
6          p.cloth = "clean";
7      }
8
9      void cook(Person p) {
10         p.cloth = "dirty";
11     }
12
13     void watchMovie(Person p) {
14         p.mood = "happy";
15     }
16 }
17

```

Figure 1: a class “Person”

is crucial because the Reinforcement Learning algorithm requires constant feedback in order to perform a dynamic decision process. Additionally, we designed and implemented a scaled reward equation for the Reinforcement learning that addresses both the time cost and precision. This reward equation will help the Reinforcement Learning agent to make the best decision on which context sensitive analysis option to use on each functions in a program.

2 Background

2.1 Basic Pointer Analysis

Pointer Analysis is the practice of identifying the properties of pointers in a program. Particularly, we would care about what are the possible values each pointer may hold in a large program, and from there, look for alias pointers, or pointers that may be pointing to the same location in memory.

Say it is a Sunday afternoon. A person Nancy plans to watch a movie, cook some food to eat, and at the very end, Nancy would like to end the day with her clothes being clean. The definition of `Person` is displayed in Figure 1. Nancy

```

1  Person nancy = new Person();
2  watchMovie(nancy);
3  cook(nancy);
4  laundry(nancy);
5

```

Figure 2: a snippet of code that describes Nancy’s Sunday afternoon plan

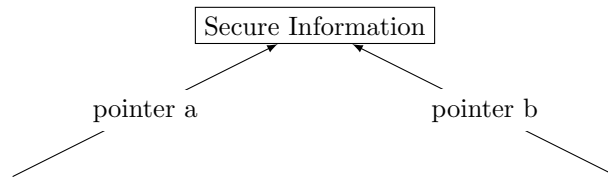


Figure 3: pointer a and pointer b are alias pointers

has decided to execute her actions as described in the code snippet in Figure 2: After watching a movie, Nancy has gained a good mood. Then she cooks, and while she is cooking, the clothes on Nancy get dirty. However, Nancy did laundry at the end, and her clothes were back to clean again. She has accomplished the goal of keeping her clothes clean and having a good mood after finishing all desired activities.

Here, the `p.cloth` pointer in the `cook` function and the `p.cloth` pointer in the `laundry` function are alias pointers, they both point to the location in memory that determines the `cloth` parameter of the `nancy` object.

Why would we care about alias pointers? There are many benefits to understanding how pointers behave in a program. For example, a large company is using a program to record its information. In the program, there is a pointer *a* that points to an important piece of secure information about this company. It is made sure that only the owner of the company has access to *a*. At the same time, this program also has a user interface for the company’s customers, where the customers would enter a certain input, and a pointer *b* would hold this inputted information. Suppose pointer *a* and pointer *b* are aliases in this

```

1  Person nancy = new Person();
2  cook(nancy);
3  laundry(nancy);
4  watchMovie(nancy);
5

```

Figure 4: Order of execution for Figure 2 after compiler pipe-lining

program, as displayed on Figure 3. Now, pointer b may also be pointing to the same memory location that holds the secure information, and data entered through the user interface may directly change that piece of information, causing severe security problems. However, if we can successfully identify that b aliases a , then we could avoid such a situation.

Moreover, identifying alias pointers can help with pipe-lining in a compiler, that is, it can change the execution order of tasks to result in a quicker completion. Again, suppose that person Nancy plans to watch a movie, cook, and do laundry. However, Nancy has an additional goal of completing all her desired activities in the shortest time possible. This can be achieved by a compiler that is capable of pipe-lining. Thus we provided it with the same program as before, as displayed in Figure 2.

In this case, if the compiler is provided with the knowledge that the `p.cloth` pointer in the `cook` function and the `p.cloth` pointer in the `laundry` function are alias pointers. Then it will not change the relative order of those two functions for `nancy` in order to finish the routine with clothes being clean, but it would be able to change around the orders of other functions. Figure 4 displays the resulting order of execution after the compiler performs pipe-lining.

This time, after Nancy puts in her laundry, she can let the washing machine do its business while enjoying the movie. This newly pipe-lined program takes less time than the original one to complete. Nancy has now finished her activities satisfactorily while achieving the goal of completing them in a shorter time

```

1  class Obj {
2      Obj value;
3
4      void f(Obj x) {
5          this.value = x;
6      }
7  }
8
9  Obj o = new Obj();
10 Obj a = new Obj();
11 Obj b = new Obj();
12 o.f(a);
13 o.f(b);
14

```

Figure 5: Example where call-site sensitivity performs better than object sensitivity

frame. This was made possible as the compiler successfully identified the alias pointer in the program.

2.2 Context Sensitive Pointer Analysis

There are sophisticated Pointer Analysis algorithms for finding alias pointers out there like the Steensgaard Algorithm [6] and Andersen Algorithm [1]. They are both bounded under $O(n^3)$ but they are only efficient when considering *context-insensitive* pointer analysis. When used with *context-sensitive* pointer analysis, they could be very inefficient in practice. Our goal is to balance the efficiency and extent of precision when performing pointer analysis with context sensitivity options. Context-sensitive analysis takes into consideration a *context* rather than just straightforwardly analyzing what each pointer could point to in a program. This “context” we are taking into consideration could be of any type, but for the analysis to be meaningful, we would look into code-related aspects like *call-site sensitivity* or *object sensitivity*. Their behavior can be clarified with an example.

If we do a context-insensitive analysis for the code present in Figure 5, we

would have the following results:

context(insensitive)	variable	heap object
$\square$	<code>o</code>	object <code>o</code>
$\square$	<code>a</code>	object <code>a</code>
$\square$	<code>b</code>	object <code>b</code>
$\square$	<code>x</code>	object <code>a</code> , object <code>b</code>
$\square$	<code>this.value</code>	object <code>a</code> , object <code>b</code>

Table 1: context insensitive analysis on Figure 5

If we analyze the pointers of this program in the context of call-site, then at lines 9, 10, and 11 we would have the pointers(or variables) `o`, `a`, and `b` pointing to a location in memory allocated for the three different new Objects (`o`, `a`, and `b`). At line 12, we call the function `f` with an input value of object `a`. Thus we have pointer `x` now pointing to the same location that pointer `a` points to, which is at object `a`. The same thing happens to the variable `this.value`, as it is assigned to the value that pointer `x` holds. And since the assignment of pointer `this.value` happened inside function `f` and function `f` is invoked on line 12, both assignments happened under the context of line 12. Similarly, pointer `x` and pointer `this.value` is assigned to a memory space again as function `f` is invoked on line 13. This time, to the location of object `b`, for pointer `b` was the given input for function `f`. Thus in the context of line 13, we have the analysis of pointers `x` and `this.value`.

context(call-site)	variable	heap object
[9]	<code>o</code>	object <code>o</code>
[10]	<code>a</code>	object <code>a</code>
[11]	<code>b</code>	object <code>b</code>
[12]	<code>x</code>	object <code>a</code>
[12]	<code>this.value</code>	object <code>a</code>
[13]	<code>x</code>	object <code>b</code>
[13]	<code>this.value</code>	object <code>b</code>

Table 2: call-site sensitive analysis on Figure 5

Now, we know that `x` and `this.value` with the context of line 12 will be

alias with **a** on line 10. On the other hand, the **x** and **this.value** with the context of line 13 will be alias with **b** in the context of line 11.

However, if we look at the same segment of the code with a different context sensitivity, the resulting conclusion may be different. Let's study it again with 1-object sensitivity, which only look at the immediate object that invokes the pointer assignment. Now, instead of caring which line point assignment occurs on, we care about the immediate object that invoked the process. Again, we begin with pointers **o**, **a**, and **b**, pointing to according locations object **o**, object **a**, and object **b**. However, since the creation of those new objects did not occur in the scope of an object, those pointers were assigned without a context, thus we will leave them blank. Things start to get interesting as we reach the usage of function **f**. Here, function **f** is first called on **o**, thus under the context of **o**, **x** and **this.value** is assigned to the same location as **a**, which is object **a**. Now as we reach the second call to function **f**, although the input is different this time, **f** is still called on **o**, thus under the same context, **x** and **this.value** can now be pointing to object **b**. Here, **x** and **this.value** are alias pointers, because they both have the possibility of pointing to the location of object **a** and object **b** under the same context.

context(1-object)	variable	heap object
[-]	o	object o
[-]	a	object a
[-]	b	object b
[o]	x	object a , object b
[o]	this.value	object a , object b

Table 3: 1-object sensitive analysis on Figure 5

Note that if we choose to consider 2-object sensitivity, we can assume that this code takes place in a function **g**. If **g** is called on an object **c**, as displayed in Figure 6, then the context of **x** and **this.value** will be [**c**,**o**].

But for now, let's just focus on the difference between call-site sensitivity and

```

1  class Obj {
2      var value
3
4      f(Obj x) {
5          this.value = x
6      }
7
8      g() {
9          Obj o = new Obj()
10         Obj a = new Obj()
11         Obj b = new Obj()
12         o.f(a)
13         o.f(b)
14     }
15 }
16
17 Obj c = new Obj()
18 c.g()
19

```

Figure 6: Example built on Figure 5 where 2-object sensitivity is performed

context(2-object)	variable	heap object
[-, -]	c	object c
[-, c]	o	object o
[-, c]	a	object a
[-, c]	b	object b
[c, o]	x	object a, object b
[c, o]	this.value	object a, object b

Table 4: 2-object sensitive analysis on Figure 6

object sensitivity in general in order to observe how different context sensitivity behaves distinctively.

In this case, analyzing the pointers with call-site sensitivity would provide more information, thus more precision. That is, although at some point **x** and **a** are aliases, if we know we are looking at the **x** whose assignment is invoked at line 5, then it would not be alias with **a**, and we no longer need to worry about encountering severe security problems.

However, different context sensitivity would perform differently on different programs. Assume we are provided with a piece of program displayed in Figure 7

```

1  class Obj {
2      var value
3
4      f(Obj x) {
5          this.h(x)
6      }
7
8      h(Obj i) {
9          this.value = i
10     }
11 }
12
13 Obj o = new Obj()
14 Obj a = new Obj()
15 Obj b = new Obj()
16 Obj c = new Obj()
17 o.f(a)
18 b.f(c)
19

```

Figure 7: Example where object sensitivity performs better than call-site sensitivity

context(call-site)	variable	heap object
[13]	o	object o
[14]	a	object a
[15]	b	object b
[16]	c	object c
[17]	x	object a
[5]	i	object a, object c
[5]	this.value	object a, object c
[18]	x	object c

Table 5: call-site sensitive analysis on Figure 7

The second call to function `f` still invokes the call of function `h` and the assignment for `i` and `this.value` at line 5, thus the context remains the same while the possible memory locations being pointed to increase.

For this program, 1-object sensitivity provides more information, thus more precision. This is because function `f` is invoked on different objects, while function `h` is repeatedly invoked on the same line (call-site).

Type sensitivity would also come up later in this paper since it is one of the key components of our project. It has a similar behavior to object sensitivity.

context(1-object)	variable	heap object
[-]	o	object o
[-]	a	object a
[-]	b	object b
[-]	c	object c
[o]	x	object a
[o]	i	object a
[o]	this.value	object a
[b]	x	object c
[b]	i	object c
[b]	this.value	object c

Table 6: 1-object sensitive analysis on Figure 7

However, when determining the context, instead of looking at the objects in the invocation chain, type sensitivity only recorded the type of the objects in the invocation chain. If we were to do a 1-type context sensitivity analysis instead of 1-object context sensitivity analysis on the previous code, we would get:

context(1-type)	variable	heap object
[-]	o	object o
[-]	a	object a
[-]	b	object b
[-]	c	object c
[Obj]	x	object a, object c
[Obj]	i	object a, object c
[Obj]	this.value	object a, object c

Table 7: 1-type sensitive analysis on Figure 7

Note that in comparison, both `o` and `b` are of the type `Obj`, thus type-analysis generates less distinct contexts than the object-analysis, this is true in any program. However, as discussed before, this is no longer true if we are looking at different numbers of objects in the call-chain.

This leads to the question: how do we know which context-sensitive analysis we should use for different programs?

3 Methodology

3.1 Scalar

There have been existing attempts to apply selective context-sensitive analysis to a program. For our project, we worked on studying and building up *Scalar* [4], a pointer analysis program for Java programs.

Scalar focuses on the “scalability” of the pointer analysis process that has been performed. To ensure that a program is scalable, Scalar allows users to input a threshold, and it does pointer analysis while keeping the program’s “time cost” under the provided constraint. In Scalar, the time is estimated by the number of contexts generated during pointer analysis, because the bottleneck for context-sensitive pointer analysis is the large number of contexts that are created.

3.2 Our Implementation

The process of Scalar doing a fully guided context analysis consists of two parts: pre-analysis and post-analysis. Scalar would first run a context-insensitive analysis of the program, generating facts like the points-to sets of the program. These facts would be used later in the post-analysis process, to compute the worst time based on the maximum number of contexts for each type of context sensitivity. Scalar uses a binary search to compute a reasonable time threshold for each function and chooses the most precise context sensitivity option that falls below the calculated threshold. The decision for which context sensitivity each function would use is made and performs the various context-sensitive analysis on the program. Scalar’s infrastructure consists of Datalog logic for pointer analysis and code specific to Scalar’s computation. The Datalog logic is written mostly in Soufflé, while the Scalar computation is written mainly in

Groovy.

3.2.1 Adding 1-obj and 3-obj Sensitivity

The published version of Scalar allows 2-object sensitive analysis, 1-type and 2-type sensitive analysis, and context-insensitive analysis. Among those options, 2-object sensitive is the most costly (in terms of time), along the line goes 2-type, 1-type, then context-insensitive. On top of that, we have added the context-sensitive options for 1-object and 3-object. Since we are analyzing Java, an object-oriented programming language, object-sensitive analysis would be one of the most fitted analysis options.

In order to add in the two additional context sensitivity analysis choices, we modified both the Soufflé code and Groovy code. First, we modified the Groovy code to calculate the worst possible number of contexts that 1-object context sensitivity and 3-object context sensitivity would possibly provide and passed the calculated information into a “Context Counter” map that then would be used in the post-analysis process. Then we dived into the pre-analysis Soufflé portion of Scalar and added relation declaration and definition for both 1-object and 3-object sensitivity. Originally, when analyzing function `f1` in the given code Figure 8, Scalar associates `f1` with a tuple, “HContextElement”, that contains two objects `o1` and `o2`. `o1` is the object on which `f1` is called, and is referred to as the receiver object of `f1`. `o2` is the receiver object of function `f2` in which `f1` is called. `o2` is referred to as the preceeder object of `f1`. Then Scalar defines the context sensitivity relations by populating the tuple. In order to support 3-object sensitivity relations, we extended the Soufflé code to keep track of an additional receiver object, `o3`, associated with every function in which `f2` is called.

For each invoked method, the call graph only contains the receiver object and the method’s invocation site from which we can directly compute the rest

```

1  void f2() {
2      o1.f1();
3  }
4
5  void f3(){
6      o2.f2();
7  }
8
9  o3.f3();
10
11

```

Figure 8: Example for the definition of HContextElement

of the 2-object sensitivity relation. For example, when analyzing `f1` in Figure 8, we use the call graph edge going from `f2` to `f1` to calculate the 2-object context $[o2, o1]$. However, this method is not generalizable for k -object sensitivity for $k > 2$.

To address this, we implemented an algorithm that utilizes the computed 2-object relation recursively to identify objects higher in the function invocation chain. This algorithm allows us to efficiently compute 3-object relations without redundancy. For example, for calculating 3-object context for `f1` in Figure 8, we use the 2-object contexts for `f1` and `f2`, $[o2, o1]$ and $[o3, o2]$. These contexts can be combined to calculate the 3-object context for `f1` because `f2` is the invocation site of `f1`, and `o2` is both the receiver object of `f2` and the preceder object of `f1`. This works because the chain of objects that invokes the functions is consistent in a program.

The process for defining the relation for 1-object sensitivity is simpler. Again, we use the already-established 2-object relation and simply keep the receiver object while ignoring the preceder object.

Providing the option for 3-object sensitivity allows a more precise analysis of a given program, and provides a deeper insight into the chain of functions called when determining the assignment of a pointer, especially with large programs, for they are very likely to have a chain of functions calls with a size more than

3 before a pointer is assigned to a location in memory. However, due to the exponential nature of the number of contexts, the use of 3-object sensitivity is prohibitively costly unless we are able to accurately identify a small number of functions for which 3-object sensitivity would drastically reduce the size of points-to sets.

On the other hand, providing the option for 1-object analysis allows having a “low-cost” option in comparison to the 2-object context-sensitive analysis, while maintaining the object information in comparison to 1-type and 2-type context-sensitive analysis.

3.2.2 1-object sensitivity VS. 2-type sensitivity

It is for sure that 1-object is guaranteed to have the same or more precision in comparison to 1-type, but there is no way to determine whether or not 1-object sensitivity is more precise when compared to 2-type sensitivity. Here, more precision means more possible generated context under a specific sensitivity (and more cost in calculation). Comparing the precision of different context sensitivities is crucial because the implementation of Scalar directly depends on the decided order of precision. To maintain the scalability threshold, Scalar assigns each function with a specific context sensitivity before the actual analysis and chooses the analysis that provides the most precision while being under the calculated threshold.

Scalar would estimate the time (by finding the upper bound) for performing the most precise analysis for every function, and check to see if the given time restraint allows the analysis to be executed. If so, it will reserve the time for the analysis decision made for this function, then move on to the next function, and so on. If not, then it will check if the next precise analysis option for this function could fit in the provided time restriction, and so on. If Scalar checks on a less precise analysis option and chooses it for a function, there may be

an analysis option that has a better performance on this function and does not violate the time threshold, however to save time, Scalar would move on after seeing that there is an option that works, and we would gain less information through the analysis when we could have gained more. Thus the order of the precision of the context sensitivity options is important.

However, the performance of 1-object context sensitivity and 2-type context sensitivity depends on the program that has been analyzed. If at the nearest levels, the function is invoked multiple times by varying objects of the same type, then 1-object context sensitivity is likely to provide more precision when analyzing this piece of the program. If the function is only called on the same objects, and those calls lie in functions that have many varying call-sites, then the 2-type context-sensitive analysis will have more advantage when analyzing this piece of the program. Therefore, without a static analysis of the types of function invocations of the program, it is very hard to tell whether or not 1-object context sensitivity or 2-type context sensitivity will yield a more precise analysis.

In our actual implementation, we have decided to prioritize 2-type context sensitivity over 1-object sensitivity. We have assumed that most programs are unlikely to have the same type of objects consecutively in the invocation chain and that most programs are more likely to have the same object calling the same function multiple times.

4 Reinforcement Learning

That being said, Scalar’s Pointer Analysis is done with one of the key components being decision-making. It would select which context sensitivity to use for each function based only on the maximum number of contexts of the function. Other pointer analysis frameworks (such as Zipper [5]) do something

similar but based on other properties of the functions. All these methods make the decision of context sensitivity for each function in isolation, however, the optimal context-sensitive option for one function can be highly dependent on the context-sensitive analysis performed on the other functions in the program. That is, the context sensitivity decisions could be better formulated using the Markov Decision Process. We have designed a Reinforcement Learning training system where the agent can learn an optimal policy for assigning a context sensitivity for a program based on the facts about the program and the effect of the context sensitivity decisions on the other parts of the program. Rewards will be given when the context-sensitive decision made for the functions in a program yields better performance in terms of time cost and the precision of the result. We would not look at memory cost because it depends on the framework of the pointer analysis, not on the decisions for which types of analysis to apply to the individual functions in a program.

In this section, each key component of the Reinforcement Learning setup will be explained along with our design and implementation details.

4.1 State

The idea is that the Reinforcement Learning agent will decide on which context sensitivity to use for a function at a time, depending on the function and the program that are being analyzed. Thus the states in our case will be the individual functions in the program that we are feeding to the pointer analysis process. More specifically, the features of the functions. For each function, there will be a list of features representing its properties. These features will be based on semantic properties rather than on syntactic properties. Those features are what the agent will use for identifying functions and calculating the best context sensitivity decision. Most of them will be generated using the tool Soot [7], a static

analysis engine, and a pre-run context-insensitive analysis of the entire program. The features would be designed to be numerical values representing call-graph relations of each function, or static identities like the amount of variables declared in the program. We would mostly focus on features that tell us more about the impact of the function on the object relations in the program since our context sensitivity options are based on object invocations. The call-graph features we use are the number of incoming and outgoing edges of a function and the depth of the function within the call-graph. Other static features we use are the sum of points-to set of variables that are declared in the function, the number of input parameters, the number of inter-procedural and local assignment statements, and the number of local, instance, and global variables used in the function. We also create a feature that describes the importance of a function, we will discuss it in Section 4.1.1.

In order to achieve the “function-by-function” infrastructure, we can still use the Soufflé code from Scalar. The program will need to be dissected into functions and be fed into the analysis process individually. Moreover, we will need to change the input and output files of the Soufflé program to allow the Reinforcement Learning to make decisions during the pointer analysis.

We will do a pre-analysis run on the original program to generate some basic facts before performing the actual Markov Decision Process. These facts are mostly what Scalar uses when performing the pointer analysis, and stay consistent throughout the analysis process. However, in order to analyze one function’s points-to set, we might need another function’s points-to set, which can only be outputted after we analyze this other function. Thus we will need to keep a dynamic record of the points-to sets of the variables in the analyzed functions. It will be part of the input files at the beginning of each analysis cycle (when entering a state). The updated version will be outputted after the

analysis of the current function is finished and inputted for the next analysis cycle.

Moreover, we will also keep a dynamic record of the decisions the agent has made on each state, and use this information for the reward calculation in the Reinforcement Learning process. However, this is not important for the actual pointer analysis process.

Previously, Scalar did not care about the order of functions when performing the analysis because it would have all the facts and information needed before performing the actual analysis, and thus it can technically perform the analysis in any order. However, this is no longer true when our methodology incorporates the Markov Decision Process. Since we are analyzing one function at a time, during each round of analysis, we could still be missing some pieces of information that could be the key to determining the points-to sets for certain variables in the function.

To avoid this, we have decided to manually input the order of functions that the agent will be working with depending on the inverse invocation order of the functions in the program. Moreover, when there are cycles in the call graph, we would follow the same order that the context-insensitive pre-analysis implemented. In this case, it is possible that the points-to sets for variables in a function that has already been analyzed could potentially be updated when analyzing other functions that are part of the cycle.

As the analysis is done on a function by function basis depending on the choice of context sensitivity picked by the agent, the information regarding the points-to sets of analyzed variables will be populated and updated to assist the analysis for functions in later analysis cycles that might depend on functions that are invoked earlier in the execution of the program. The order for invocation of the functions in a program could also be generated by Soot, which is built in

the pre-analysis process.

4.1.1 Score

Scalar focuses on making decisions based on the time importance of the function, which guarantees an upper bound on the scalability of the analysis process by using faster analysis for functions that require a large time cost when being analyzed precisely. It is possible that when a function is analyzed sensitively, the process will be very costly in time, but can result in a significant increase in the precision of the overall analysis. In this case, Scalar would still sacrifice the gain in precision to ensure the scalability of the analysis because Scalar uses the same time threshold for all functions in the program. We agree that the time importance is a key aspect of pointer analysis, but we also think that the effect on the overall precision should be taken into consideration. We have implemented a special feature which we named “score”. It is a numerical value generated from a combined analysis of the function’s contributions to the scalability and precision of the analysis.

After incorporating the Markov Decision Process into Pointer Analysis, we no longer have thresholds for functions the way Scalar does, because the agent should not be making decisions under a strict scalability constraint. However, it could still be helpful to identify an upper bound of the cost of analyzing a function with the possible analysis options. We decided to keep part of Scalar’s infrastructure, extended to 1-object and 3-object sensitivity, that calculates how much context each context analysis could generate for the specific function, and use this information to indicate the scalability of a function in the score feature.

In this context, we mostly define “precision” by how small a points-to set is. Since our analysis is sound, the smaller the points-to set, the more useful our pointer analysis is. Since our analysis options are based on object invocation, the more variables declared in a function and the larger the points to sets for

```

1  void f() {
2      Obj x1 = null;
3      Obj x2 = g(x1);
4      Obj x3 = g(x1);
5      Obj x4 = g(x1);
6  }
7
8  void g(Obj input) {
9      if (input) {
10         return input;
11     }
12     return new Obj();
13 }
14

```

Figure 9: Example of a function that determines a large subset of another function

these variables, the more impact this function has on the precision of analyzing the entire program. We want to identify those functions and analyze them more sensitively to reduce the imprecision introduced by them. However, there is a catch in this logic.

In Figure 9, by our current definition, function `f` declares more variables, thus it should be more important than function `g` in our analysis. However, most of the variables declared in function `f` have their meaningful value defined in function `g`. In order to gain more precision for the program analysis, we would need to treat function `g` with more sensitivity, or else we could lose some important information. Thus in our “precision” evaluation, we would need to identify those functions that contribute a large subset of the points-to sets for variables declared in other functions.

We did this by first identifying the variables that are in the top 10% in the program in terms of the size of their points-to set. Then we counted how many of those top 10% variables are declared in each function. The more there are, the more value we add to the “precision” part of this function’s score feature, indicating that it might be important to analyze this function more sensitively. When function `f` declares many variables with large points-to sets,

the majority of the heap objects in those points-to sets are likely being defined in other functions and passed to the variables in function `f` through either return variables or parameter values. Thus we also identified the top 10% of parameters and return variables in terms of points-to set size, and gave more weight to the determination of the “precision” of a function that contains them.

After we determine the scalability and the precision of a function, we can calculate the score of the function by balancing the two factors. The value of scalability and precision is normalized for they are computed on different scales, and we also adjusted the parameters of the score equation to avoid negative scores and extremely large numbers.

In general, the score feature represents the trade-off between the cost it can take to analyze a function and the corresponding gain(precision) that comes with it. This customized feature depends not only on the function itself but on the entire program that the function is in since we use the points-to sets information generated from the context-insensitive pre-analysis. Score is engineered into the analysis process thus it will be generated along with the other fact files when we run the pre-analysis before performing the Reinforcement Learning training.

4.2 Action, Environment, and Agent

The action definition for our Reinforcement Learning process is pretty intuitive. The action would be choosing the context sensitivity analysis for each function in the program. The agent would have six context sensitivity options: context insensitive, 1-type context sensitivity, 2-type context sensitivity, 1-object context sensitivity, 2-object context sensitivity, and 3-object context sensitivity. Here, a type sensitivity is very similar to an object sensitivity, the only difference is that instead of treating the specific object that invoked a certain function as its context, we would only look at the type of the object that did

the invocation.

Usually, the environment in an Reinforcement Learning process refers to a set of unchanging conditions that simulate the problem that we are trying to solve. Here, the environment of our Reinforcement Learning design would be the Doop [2] framework for Pointer Analysis, where we have done some modifications so it can serve our function-by-function cycle analysis. We will mostly rely on its ability to perform a points-to analysis with the designated context sensitivity options. We will use DaCapo benchmarks [3] for training and testing purposes.

We are going to formulate the Markov Decision Process when training our agents, and we will try out different agents available on the market and compare their performances.

4.3 Reward

Since the order of functions is determined manually, this grants the agent’s actions to be episodic tasks. Once we have performed pointer analysis on all functions in a program, the state terminates and the decision process comes to an end. In our reward function, we only care about two factors, the time cost and the amount of information that was generated (precision) from the agent’s decision. For the time cost, we will directly use the run-time that is recorded by the system for performing pointer analysis on each function after the agent makes its decision. However, since the points-to set of one function may depend on the analysis result for another function, for the precision gain we can get from taking a certain action, we will look at how many cumulative points-to relations ignoring duplicate relations due to context there are in the current analysis. We will not only count the new points-to relations generated after transitioning to the most recent state, but also the entire state including point-to relations

generated from previously analyzed functions.

It is crucial to evaluate the reward of different decisions on a fixed scale. We would run a context-insensitive pre-analysis and use its results as a standard scale. We would refer to the ratio between the results from the pre-analysis process and the Reinforcement Learning process. One challenge which defining the reward based on precision in the function-by-function analysis is that some variables may have small points-to set when their declaring functions are analyzed early in the process, however, those points-to set may increase dramatically when other functions are analyzed later in the process. To avoid having functions penalized for appearing later in the analysis order, we decided to implement an incremental version of the context-insensitive pre-analysis using the same function order as the Reinforcement Learning process. For the precision aspect of the reward, we compare the points-to set of the current state to the points-to set of the corresponding intermediate state in the context-insensitive pre-analysis.

We will use the value of the ratio for the two factors, time cost and the size of the points-to set, between the Reinforcement Learning process result and the pre-analysis. Ideally, lower time costs and more precision would grant the agent a higher reward after taking an action. Thus the design of our reward function would be:

$$R(S_i | A) = \left(\frac{T_{RL}}{T_{pre-analysis}} \right) - \left(\frac{VPTsize_{RL}}{VPTsize_{pre-analysis}} \right)$$

Here, T_{RL} is the time of execution for the current function in the agent-chosen analysis, $T_{pre-analysis}$ is the time of execution for the current function in insensitive analysis, $VPTsize_{RL}$ is the cumulative size of Variable Points To sets in the agent-chosen analysis, and $VPTsize_{pre-analysis}$ is the cumulative size of Variable Points To sets in insensitive analysis.

The goal for the agent would be to get the highest possible cumulative reward

at the terminal state. It would be able to learn from the reward feedback to know which context sensitivity option for functions with certain features is the most rewarding.

5 Conclusion

This project initially started aiming to modify Scalar so that it determines the time threshold differently. Scalar uses the same time threshold for all functions, and this algorithm tends to sacrifice the precision gain of an analysis to ensure the scalability of a program. We wanted to see how Scalar’s performance would improve if the time threshold is defined differently for functions based on the function’s semantic properties. However, as we progressed, we found that the idea of incorporating a Markov Decision Process would be more promising. Rather than treating scalability as a constraint, the Markov Decision Process would take both scalability and precision of the pointer analysis process into consideration and grants more weight to precision evaluation.

Our next step would be running the Reinforcement Learning process on the pointer analysis framework that we modified. We would be likely to try out different agents that are available on the market and compare and contrast the results. Once we have some data to analyze, we might also adjust the reward function if needed. We will likely dissect the large DaCapo java benchmarks and train the agent using smaller chunks of code while using the rest of the code parts for development testing. We may also permute some parts of the code, especially objects invocation functions and functions that serves as invocation site, so that we can work with variants of test data.

References

- [1] Lars Ole Andersen and Peter Lee. Program analysis and specialization for the c programming language. 2005.
- [2] Tony Antoniadis, Konstantinos Triantafyllou, and Yannis Smaragdakis. Porting doop to soufflé: A tale of inter-engine portability for datalog-based analyses. In *Proceedings of the 6th ACM SIGPLAN International Workshop on State Of the Art in Program Analysis*, SOAP 2017, page 25–30, New York, NY, USA, 2017. Association for Computing Machinery.
- [3] Stephen M. Blackburn, Robin Garner, Chris Hoffmann, Asjad M. Khang, Kathryn S. McKinley, Rotem Bentzur, Amer Diwan, Daniel Feinberg, Daniel Frampton, Samuel Z. Guyer, Martin Hirzel, Antony Hosking, Maria Jump, Han Lee, J. Eliot B. Moss, Aashish Phansalkar, Darko Stefanović, Thomas VanDrunen, Daniel von Dincklage, and Ben Wiedermann. The dacapo benchmarks: Java benchmarking development and analysis. In *Proceedings of the 21st Annual ACM SIGPLAN Conference on Object-Oriented Programming Systems, Languages, and Applications*, OOPSLA '06, page 169–190, New York, NY, USA, 2006. Association for Computing Machinery.
- [4] Yue Li, Tian Tan, Anders Møller, and Yannis Smaragdakis. Scalability-first pointer analysis with self-tuning context-sensitivity. In *Proceedings of the 2018 26th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*, ES-EC/FSE 2018, page 129–140, New York, NY, USA, 2018. Association for Computing Machinery.
- [5] Yue Li, Tian Tan, Anders Møller, and Yannis Smaragdakis. A principled approach to selective context sensitivity for pointer analysis. *ACM Trans. Program. Lang. Syst.*, 42(2), may 2020.

- [6] Bjarne Steensgaard. Points-to analysis in almost linear time. In *Proceedings of the 23rd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages*, POPL '96, page 32–41, New York, NY, USA, 1996. Association for Computing Machinery.
- [7] Raja Vallée-Rai, Phong Co, Etienne Gagnon, Laurie Hendren, Patrick Lam, and Vijay Sundaresan. Soot - a java bytecode optimization framework. In *Proceedings of the 1999 Conference of the Centre for Advanced Studies on Collaborative Research*, CASCON '99, page 13. IBM Press, 1999.